ЛАБОРАТОРНА РОБОТА № 4

Візуальний аналіз даних та часові ряди — аналіз у двох середовищах

Курс: СК26 «Інтелектуальний аналіз та візуалізація даних» **Кафедра систем штучного інтелекту, Національний університет «Львівська політехніка»**
Оцінювання: 4 бали

Визначення варіанта

Заповніть три поля в клітинці нижче й виконайте обидві службові клітинки — номер варіанта, ринок і товар визначаться автоматично.

In [1]:
```
# ── Заповніть три поля ──────────────────────────────────────────
SURNAME_NAME = 'Басараб Дмитрій'      # прізвище та ім'я
GROUP        = 'СШІ-32'      # шифр групи, наприклад СШІ-31
NUMBER       = 1       # ваш номер за списком групи (1–30)

assert SURNAME_NAME.strip(), 'Впишіть прізвище та ім\'я'
assert GROUP.strip(),        'Впишіть групу'
assert isinstance(NUMBER, int) and 1 <= NUMBER <= 30, 'Номер має бути цілим від
print('Дані прийнято:', SURNAME_NAME, '|', GROUP, '| номер', NUMBER)
```

```
Дані прийнято: Басараб Дмитрій | СШІ-32 | номер 1
```

In [2]:
```
# ── Службова клітинка: визначення варіанта. Змінювати не потрібно ──
import json
"""
"варіант": 1,
  "ринок": "Odesa",
  "товар": "Bread (wheat) - Retail",
  "товар_укр": "хліб пшеничний"
"""
VARIANTS = json.load(open('lr4_variants_student.json', encoding='utf-8'))
V = VARIANTS[NUMBER - 1]

MARKET, GOOD, GOOD_UA = V['ринок'], V['товар'], V['товар_укр']

print('=' * 72)
print(f'ВАРІАНТ {V["варіант"]}   ·   {SURNAME_NAME}   ·   {GROUP}')
print('=' * 72)
print()
print(f'   Ринок:  {MARKET}')
print(f'   Товар:  {GOOD_UA}   (у наборі даних: {GOOD})')
print()
print('Далі в роботі користуйтеся змінними MARKET і GOOD —')
print('тоді весь код автоматично працюватиме саме з вашим рядом.')
print()
print('Період: 03.2014 – 12.2017 · очікувана довжина ряду: 46 місяців')
print('Фолдів ретроспективної перевірки: 7 · горизонт: 3 місяці · ARIMA(1, 1, 1)
```

```
======================================================================
ВАРІАНТ 1   ·   Басараб Дмитрій   ·   СШІ-32
======================================================================

   Ринок:  Odesa
   Товар:  хліб пшеничний   (у наборі даних: Bread (wheat) - Retail)

Далі в роботі користуйтеся змінними MARKET і GOOD —
тоді весь код автоматично працюватиме саме з вашим рядом.

Період: 03.2014 – 12.2017 · очікувана довжина ряду: 46 місяців
Фолдів ретроспективної перевірки: 7 · горизонт: 3 місяці · ARIMA(1, 1, 1)
```

> ⚠️ Якщо ці дві клітинки не виконано, усі подальші завдання не мають сенсу: перевіряти роботу будуть **за вашим варіантом**, а не за тим рядом, який ви оберете самі.

Дата здачі
Підпис

Мета роботи

Виконати повний цикл аналізу одного часового ряду **у двох середовищах** — у Python та в Orange — і пояснити, чому однаково названі метрики в них дають різні числа.

Вихідні дані

Файл	Зміст
`ціни_україна_2014_2017.csv`	Роздрібні ціни на 25 товарів по 28 ринках України, щомісяця, 03.2014 – 12.2017
`lr4_variants_student.json`	Розподіл варіантів (зчитується автоматично)
`ЛР4_помилковий_аналіз.ipynb`	Звіт колеги, який треба відрецензувати (завдання 8)

Це той самий набір, з яким ми працювали на лекції 4. **Національне середнє в завданнях не використовується** — ваш ряд це конкретний обласний ринок.

Що здавати

1. **`ЛР4_Прізвище_Варіант.ipynb`** — цей зошит із вашим кодом, графіками та письмовими висновками.
2. **`ряд_для_orange.csv`** — експортований ряд (завдання 9).
3. **`потік_orange.ows`** — збережений потік Orange.

4. **Знімки екрана** з Orange для завдань 10–13 — вставте їх у зошит або додайте окремими файлами.

Правила

- **Код пишете самостійно** — у цьому зошиті навмисно немає заготовок.
- **Користуватися штучним інтелектом дозволено**, але ви зобов'язані заповнити декларацію в кінці зошита.
- **Кожне число має супроводжуватися висновком словами.** Робота, у якій є графіки й таблиці, але немає письмових відповідей на поставлені питання, оцінюється як невиконана.
- Усі обчислення мають бути **відтворюваними**: зошит має виконуватися від початку до кінця командою *Restart & Run All* без помилок.

ЧАСТИНА 1. Аналіз засобами Python

Виконується аудиторно.

Завдання 1. Підготовка ряду

Витягніть із набору ряд свого варіанта — скористайтеся змінними `MARKET` і `GOOD`, які визначилися у службовій клітинці, — і приведіть його до **повної місячної сітки** з березня 2014 по грудень 2017.

У звіті наведіть:

1. скільки місяців у сітці та скільки з них мають вимір;
2. у якому саме місяці пропуск;
3. **чому** цей пропуск виник — і чи він у вашому ряді один.

> 💡 **Підказка.** Пропуск є в усіх без винятку рядах набору. Якщо ви знайшли більше одного — перевірте, чи правильно побудована сітка.

In [3]:
```
# Завдання 1. Підготовка ряду
import pandas as pd
import numpy as np
df = pd.read_csv('ціни_україна_2014_2017.csv')
bread_na = df[(df['товар'] == GOOD) & (df['ринок'] == MARKET)]

bread_na['ts'] = pd.to_datetime({
    'year': bread_na['рік'],
    'month': bread_na['місяць'],
    'day': 1
})
bread_na
```

Out[3]:

	ринок	товар	одиниця	валюта	рік	місяць	ціна	ts
2964	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	3	4.5000	2014-03-01
2965	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	4	4.6900	2014-04-01
2966	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	5	4.6900	2014-05-01
2967	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	6	5.1900	2014-06-01
2968	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	7	5.1900	2014-07-01
2969	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	8	5.1900	2014-08-01
2970	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	9	5.2933	2014-09-01
2971	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	10	5.5000	2014-10-01
2972	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	11	5.5000	2014-11-01
2973	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	12	5.6267	2014-12-01
2974	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	1	5.9100	2015-01-01
2975	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	2	6.2700	2015-02-01
2976	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	3	8.2500	2015-03-01
2977	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	4	8.3967	2015-04-01
2978	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	5	8.4700	2015-05-01
2979	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	6	8.4700	2015-06-01
2980	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	7	8.4700	2015-07-01
2981	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	8	8.4700	2015-08-01
2982	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	9	8.4700	2015-09-01
2983	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	10	8.4700	2015-10-01
2984	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	11	8.4700	2015-11-01

	ринок	товар	одиниця	валюта	рік	місяць	ціна	ts
2985	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	12	8.4700	2015-12-01
2986	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	1	8.4700	2016-01-01
2987	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	2	8.4700	2016-02-01
2988	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	3	8.4700	2016-03-01
2989	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	4	8.5833	2016-04-01
2990	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	5	9.0500	2016-05-01
2991	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	6	9.3167	2016-06-01
2992	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	7	9.3900	2016-07-01
2993	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	8	9.5000	2016-08-01
2994	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	9	9.7500	2016-09-01
2995	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	10	8.2500	2016-10-01
2996	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	11	8.2500	2016-11-01
2997	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	12	9.7500	2016-12-01
2998	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	1	10.4167	2017-01-01
2999	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	2	10.7500	2017-02-01
3000	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	3	10.7500	2017-03-01
3001	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	4	10.7500	2017-04-01
3002	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	5	10.7500	2017-05-01
3003	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	6	10.7500	2017-06-01
3004	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	7	9.4600	2017-07-01
3005	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	9	9.4600	2017-09-01

	ринок	товар	одиниця	валюта	рік	місяць	ціна	ts
3006	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	10	9.6300	2017-10-01
3007	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	11	9.6300	2017-11-01
3008	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	12	9.6667	2017-12-01

Висновок до завдання 1:

пропущено 8 місяць 2017 і 1-3 2014, у сітці 44 місяці

Завдання 2. Заповнення пропуску

Заповніть пропуск. Порівняйте **щонайменше три** способи: перенесення останнього значення, лінійну інтерполяцію та ще один на ваш вибір.

У звіті наведіть:

1. значення, яке дає кожен спосіб;
2. який спосіб ви обрали і **чому саме його**;
3. за якої умови заповнювати пропуск було б **неприпустимо** — наведіть приклад із цього ж набору даних.

> 💡 **Підказка.** Третій пункт стосується не вашого ряду, а принципу. Згадайте, які області зникають із набору у 2014 році.

In [4]:
```
# Завдання 2. Заповнення пропуску
new_row = pd.DataFrame([{
    'ринок': 'Odesa',
    'товар': 'Bread (wheat) - Retail',
    'одиниця': 'Loaf',
    'валюта': 'UAH',
    'рік': 2017,
    'місяць': 8,
    'ціна': np.nan,  # залишаємо ціну порожньою (NaN) для інтерполяції
    'ts': pd.to_datetime('2017-08-01')
}])

# 2. Додаємо цей рядок у ваш датафрейм (замініть df на назву вашої змінної)
bread = pd.concat([bread_na, new_row], ignore_index=True)
bread = bread.sort_values(by=['рік', 'місяць']).reset_index(drop=True)
#bread.ffill()
bread['ціна'] = bread['ціна'].interpolate()
#bread = bread.fillna(bread.loc[bread['рік'] == 2017,'ціна'].mean())
bread
```

Out[4]:

	ринок	товар	одиниця	валюта	рік	місяць	ціна	ts
0	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	3	4.5000	2014-03-01
1	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	4	4.6900	2014-04-01
2	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	5	4.6900	2014-05-01
3	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	6	5.1900	2014-06-01
4	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	7	5.1900	2014-07-01
5	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	8	5.1900	2014-08-01
6	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	9	5.2933	2014-09-01
7	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	10	5.5000	2014-10-01
8	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	11	5.5000	2014-11-01
9	Odesa	Bread (wheat) - Retail	Loaf	UAH	2014	12	5.6267	2014-12-01
10	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	1	5.9100	2015-01-01
11	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	2	6.2700	2015-02-01
12	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	3	8.2500	2015-03-01
13	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	4	8.3967	2015-04-01
14	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	5	8.4700	2015-05-01
15	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	6	8.4700	2015-06-01
16	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	7	8.4700	2015-07-01
17	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	8	8.4700	2015-08-01
18	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	9	8.4700	2015-09-01
19	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	10	8.4700	2015-10-01
20	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	11	8.4700	2015-11-01

	ринок	товар	одиниця	валюта	рік	місяць	ціна	ts
21	Odesa	Bread (wheat) - Retail	Loaf	UAH	2015	12	8.4700	2015-12-01
22	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	1	8.4700	2016-01-01
23	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	2	8.4700	2016-02-01
24	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	3	8.4700	2016-03-01
25	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	4	8.5833	2016-04-01
26	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	5	9.0500	2016-05-01
27	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	6	9.3167	2016-06-01
28	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	7	9.3900	2016-07-01
29	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	8	9.5000	2016-08-01
30	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	9	9.7500	2016-09-01
31	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	10	8.2500	2016-10-01
32	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	11	8.2500	2016-11-01
33	Odesa	Bread (wheat) - Retail	Loaf	UAH	2016	12	9.7500	2016-12-01
34	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	1	10.4167	2017-01-01
35	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	2	10.7500	2017-02-01
36	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	3	10.7500	2017-03-01
37	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	4	10.7500	2017-04-01
38	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	5	10.7500	2017-05-01
39	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	6	10.7500	2017-06-01
40	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	7	9.4600	2017-07-01
41	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	8	9.4600	2017-08-01

	ринок	товар	одиниця	валюта	рік	місяць	ціна	ts
42	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	9	9.4600	2017-09-01
43	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	10	9.6300	2017-10-01
44	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	11	9.6300	2017-11-01
45	Odesa	Bread (wheat) - Retail	Loaf	UAH	2017	12	9.6667	2017-12-01

Висновок до завдання 2:

Вставлення попереднього 9.4600 інтерполяція 9.4600 середнє(взяв за 2017) 10.1800

обираю інтерполяцію середнє показало штучнй стрибок даних яке врахувало зимові ціни на хліб. але не врахувало сезонність, а інтерполяці врахувала схожі періоди

зник крим і для нього обраховувати ціну на хліб на основі довоєнних даних не можна бо не буде правдоподібним

Завдання 3. Візуальна розвідка

Побудуйте **не менше трьох** графіків свого ряду, які відповідають на різні питання.

У звіті наведіть:

1. для кожного графіка — яке питання він ставить і **чому обрано саме таку геометрію та шкалу**;
2. одну знахідку, якої ви не очікували до побудови графіків.

> 💡 **Підказка.** Графік без пояснення вибору форми не зараховується. Нагадування з лекції: геометрія — це твердження про дані.

In [5]:
```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np


# 1. Графік для циклічної шкали (місяць)

plt.figure(figsize=(10, 4))
pivot_bread = bread.pivot_table(index='рік', columns='місяць', values='ціна', ag

sns.heatmap(pivot_bread, annot=True, fmt=".1f", cmap="YlOrRd")
plt.title('Теплова карта цін на хліб (Циклічна шкала місяців)')
plt.xlabel('Місяць')
plt.ylabel('Рік')
plt.show()
```

```
# 2. Графік для інтервальної шкали (рік, дата)

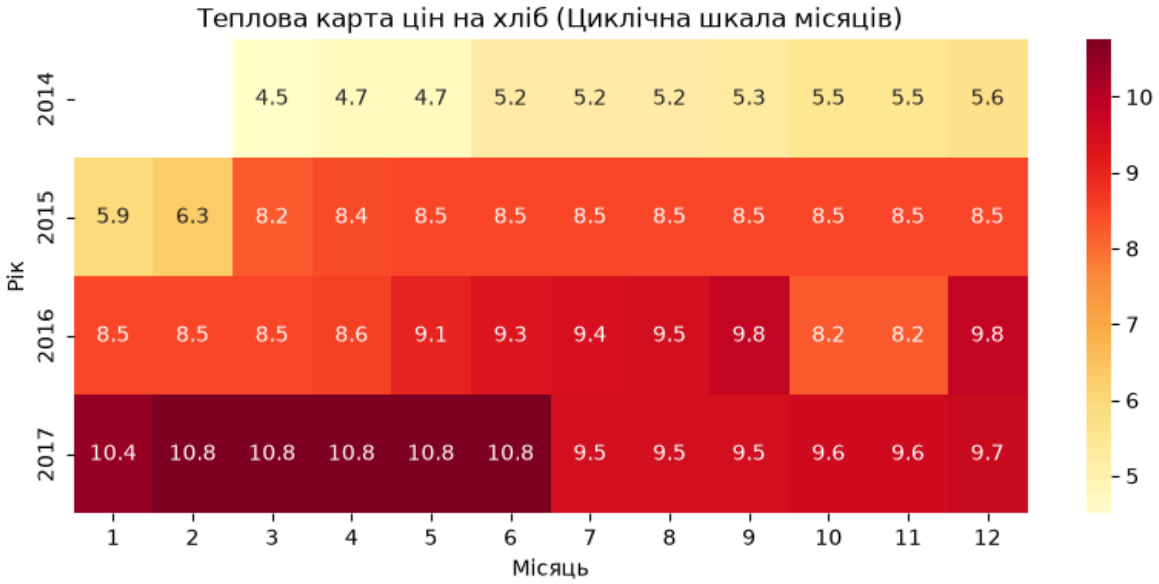
plt.figure(figsize=(8, 4))
yearly_prices = bread.groupby('рік')['ціна'].mean()

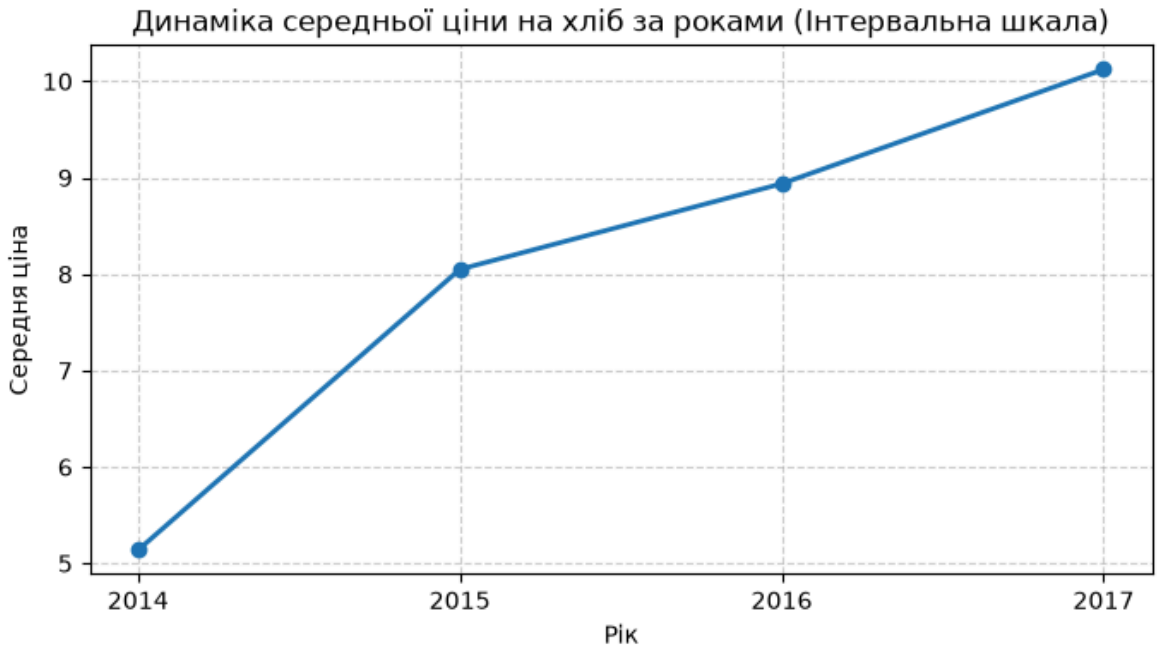
plt.plot(yearly_prices.index, yearly_prices.values, marker='o', color='tab:blue'
plt.xticks(yearly_prices.index)
plt.grid(True, linestyle='--', alpha=0.6)
plt.title('Динаміка середньої ціни на хліб за роками (Інтервальна шкала)')
plt.xlabel('Рік')
plt.ylabel('Середня ціна')
plt.show()


# 3. Графік для шкали відношень (ціна)

plt.figure(figsize=(8, 5))


sns.boxplot(data=bread, x='рік', y='ціна', palette='Set2')
plt.title('Розподіл та розкид цін на хліб по роках (Шкала відношень)')
plt.xlabel('Рік')
plt.ylabel('Ціна')
plt.show()
```

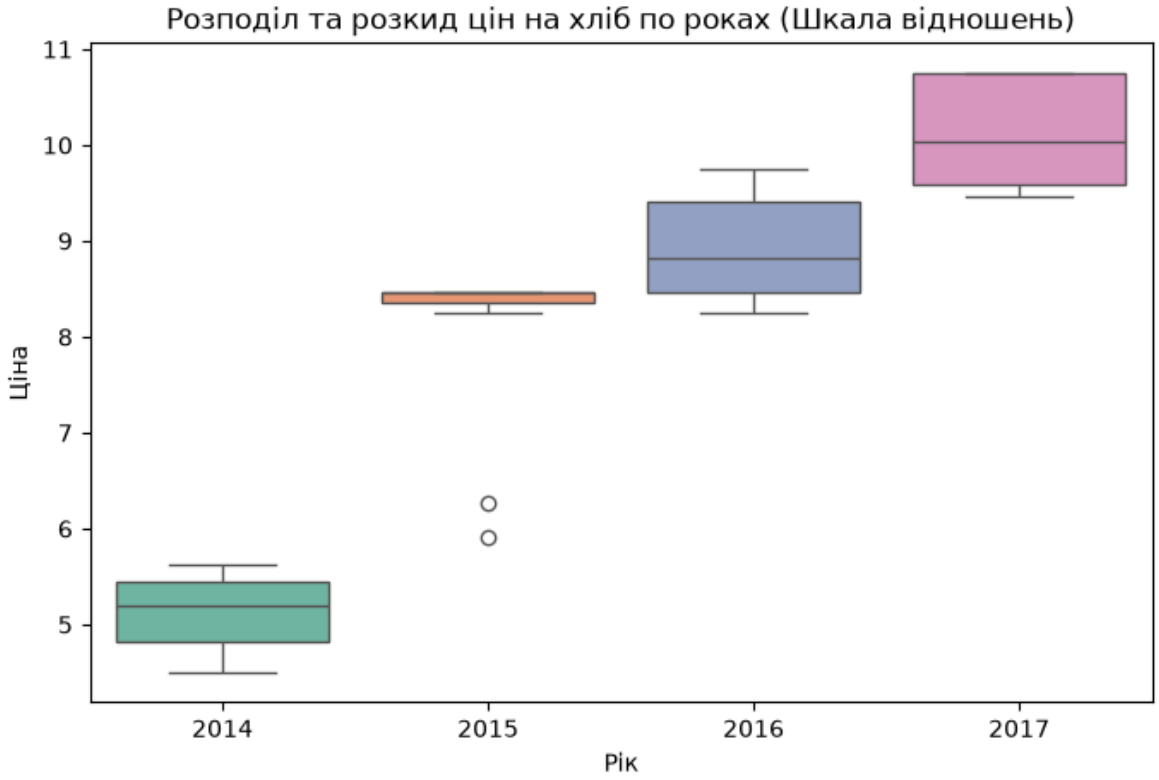





```
C:\Users\Acer\AppData\Local\Temp\ipykernel_17676\91573658.py:37: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.boxplot(data=bread, x='рік', y='ціна', palette='Set2')
```



Висновок до завдання 3: перший графік показує ціну по місяцях видно шо в сезон виросту пшениці ціна найменша другий шо ціна зростає третій показує розкид цін

Завдання 4. Розкладання на складові

Розкладіть ряд на тренд, сезонну складову та залишок методом STL із періодом 12.

У звіті наведіть:

1. чотирипанельний графік розкладання;
2. числові значення сили сезонності F_S та сили тренду F_T;
3. **висновок: чи є у вашому ряді сезонність**, із посиланням на порогове значення.

> 💡 **Підказка.** STL поверне сезонну складову завжди. Ваше завдання — вирішити, чи вона означає щось, а не чи вона є.

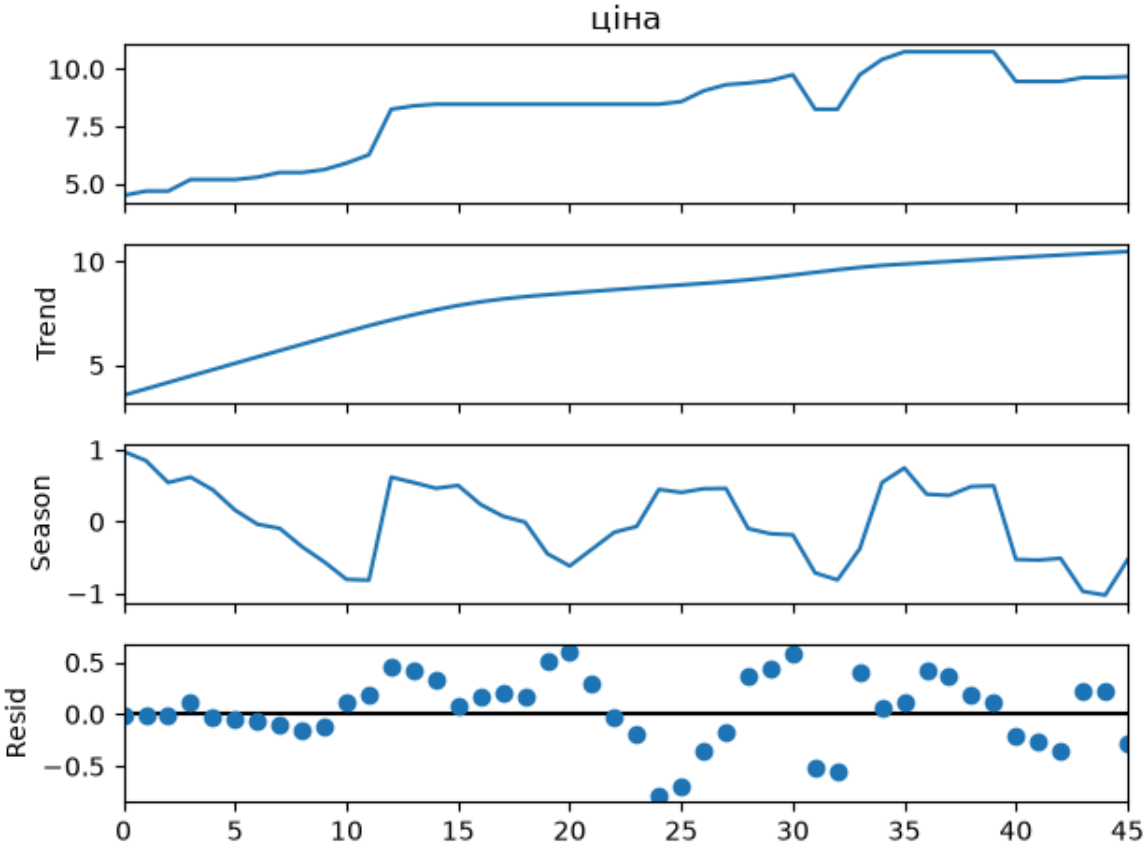
In [6]:
```
# Завдання 4. Розкладання на складові
from statsmodels.tsa.seasonal import STL
stl = STL(bread['ціна'], period=12)
res = stl.fit()
res.plot()
T = res.trend
S = res.seasonal
R = res.resid


var_R = np.var(R)
var_T_R = np.var(T + R)
var_S_R = np.var(S + R)

strength_trend = max(0.0, 1.0 - (var_R / var_T_R))
strength_seasonal = max(0.0, 1.0 - (var_R / var_S_R))

print(f"Сила тренду (Ft):     {strength_trend:.4f}")
print(f"Сила сезонності (Fs): {strength_seasonal:.4f}")
```

```
Сила тренду (Ft):     0.9740
Сила сезонності (Fs): 0.7404
```



Висновок до завдання 4:

Сила тренду (Ft): 0.9740 Сила сезонності (Fs): 0.7404 > 0.5 тому пора року означає шось при прогнозуванні, сезон не є шумом

Завдання 5. Стаціонарність

Побудуйте ACF і PACF вихідного ряду та його першої різниці. Застосуйте тести ADF і KPSS до обох.

У звіті наведіть:

1. графіки ACF і PACF до та після диференціювання;
2. таблицю з p-значеннями обох тестів для обох рядів;
3. висновок про стаціонарність **із зазначенням, узгоджуються тести між собою чи ні**;
4. якщо не узгоджуються — ваше пояснення, чому.

> 💡 **Підказка.** Нульові гіпотези ADF і KPSS протилежні. Перш ніж писати висновок, перевірте, що ви не переплутали напрямок.

In [7]:
```
# Завдання 5. Стаціонарність
import matplotlib.pyplot as plt
import pandas as pd
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.stattools import adfuller, kpss
```

```
y = bread_na["ціна"].dropna()


y_diff = y.diff().dropna()

# --- ПОБУДОВА ГРАФІКІВ ACF та PACF ---
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Вихідний ряд
plot_acf(y, ax=axes[0, 0], title="ACF: Вихідний ряд")
plot_pacf(y, ax=axes[0, 1], title="PACF: Вихідний ряд")

# Перша різниця
plot_acf(y_diff, ax=axes[1, 0], title="ACF: Перша різниця (d=1)")
plot_pacf(y_diff, ax=axes[1, 1], title="PACF: Перша різниця (d=1)")

plt.tight_layout()
plt.show()

# --- ПРОВЕДЕННЯ ТЕСТІВ ADF ТА KPSS ---



def get_test_results(series):
    # ADF тест
    adf_res = adfuller(series, autolag="AIC")
    adf_p = adf_res[1]

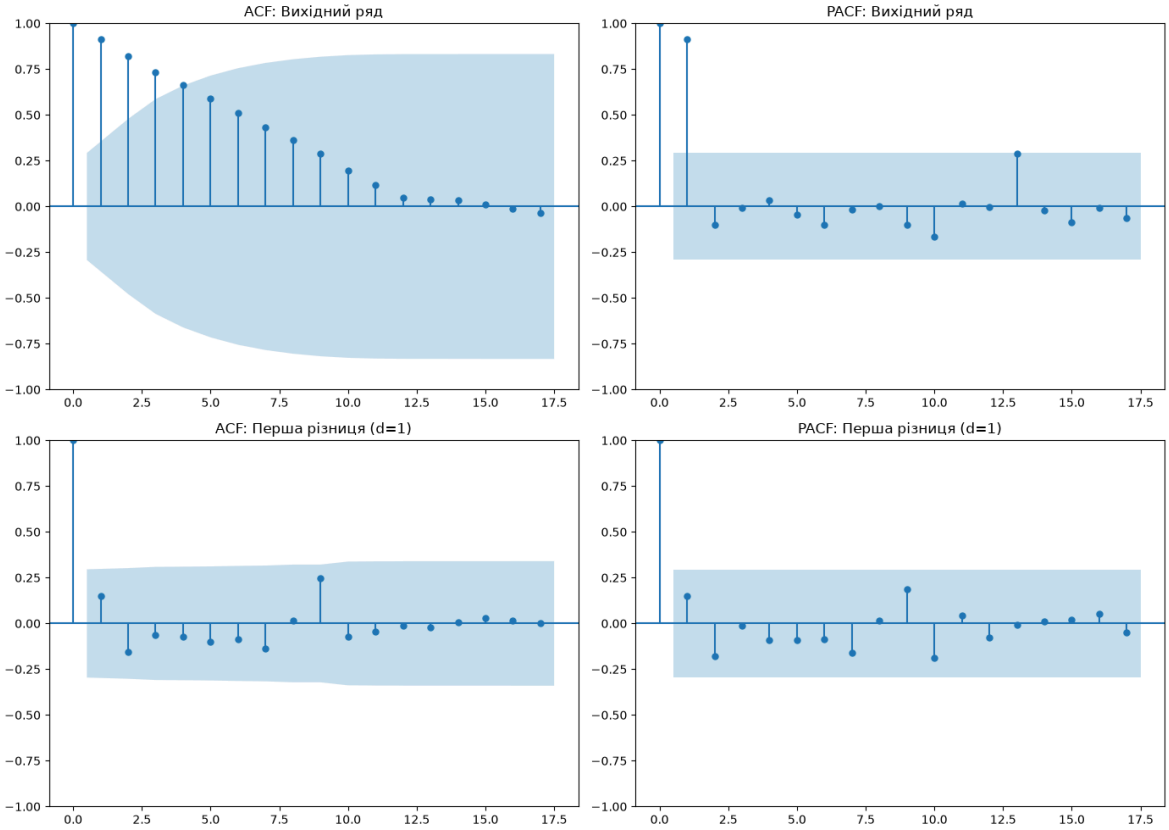

    kpss_res = kpss(series, regression="c", nlags="auto")
    kpss_p = kpss_res[1]

    return adf_p, kpss_p


adf_orig, kpss_orig = get_test_results(y)
adf_diff, kpss_diff = get_test_results(y_diff)


results_df = pd.DataFrame(
    {
        "Тест ADF (p-value)": [adf_orig, adf_diff],
        "Тест KPSS (p-value)": [kpss_orig, kpss_diff],
    },
    index=["Вихідний ряд", "Перша різниця (d=1)"],
)

print("\n--- ТАБЛИЦЯ P-ЗНАЧЕНЬ ---")
print(results_df.round(4))
```



--- ТАБЛИЦЯ P-ЗНАЧЕНЬ ---

	Тест ADF (p-value)	Тест KPSS (p-value)
Вихідний ряд	0.3819	0.01
Перша різниця (d=1)	0.0000	0.10

```
C:\Users\Acer\AppData\Local\Temp\ipykernel_17676\3046729303.py:33: FutureWarning:
adfuller currently returns a plain tuple whose length depends on the store and au
tolag arguments. In release 0.16 or after July 2027, whichever is later, the defa
ult behavior will switch to always returning an ADFullerResult. Set result_object
=True to switch now, or result_object=False to keep the current behavior and sile
nce this warning.
  adf_res = adfuller(series, autolag="AIC")
C:\Users\Acer\AppData\Local\Temp\ipykernel_17676\3046729303.py:37: InterpolationW
arning: The test statistic is outside of the range of p-values available in the l
ook-up table. The actual p-value is smaller than the p-value returned.
  kpss_res = kpss(series, regression="c", nlags="auto")
C:\Users\Acer\AppData\Local\Temp\ipykernel_17676\3046729303.py:37: FutureWarning:
kpss currently returns a plain tuple whose length and layout depends on the store
argument (and which silently drops `lags` when store=True). In release 0.16 or af
ter July 2027, whichever is later, the default behavior will switch to always ret
urning a KPSSResult. Set result_object=True to switch now, or result_object=False
to keep the current behavior and silence this warning.
  kpss_res = kpss(series, regression="c", nlags="auto")
C:\Users\Acer\AppData\Local\Temp\ipykernel_17676\3046729303.py:33: FutureWarning:
adfuller currently returns a plain tuple whose length depends on the store and au
tolag arguments. In release 0.16 or after July 2027, whichever is later, the defa
ult behavior will switch to always returning an ADFullerResult. Set result_object
=True to switch now, or result_object=False to keep the current behavior and sile
nce this warning.
  adf_res = adfuller(series, autolag="AIC")
C:\Users\Acer\AppData\Local\Temp\ipykernel_17676\3046729303.py:37: InterpolationW
arning: The test statistic is outside of the range of p-values available in the l
ook-up table. The actual p-value is greater than the p-value returned.
  kpss_res = kpss(series, regression="c", nlags="auto")
C:\Users\Acer\AppData\Local\Temp\ipykernel_17676\3046729303.py:37: FutureWarning:
kpss currently returns a plain tuple whose length and layout depends on the store
argument (and which silently drops `lags` when store=True). In release 0.16 or af
ter July 2027, whichever is later, the default behavior will switch to always ret
urning a KPSSResult. Set result_object=True to switch now, or result_object=False
to keep the current behavior and silence this warning.
  kpss_res = kpss(series, regression="c", nlags="auto")
```

Висновок до завдання 5:

ряд є нестаціонарним і kpss <0.05 підтверджує похідна показує протилежне

Завдання 6. Базисні прогнози та ретроспективна перевірка

Побудуйте **чотири** базисні прогнози: наївний, сезонний наївний, середнє та дрейф. Оцініть їх **ретроспективною перевіркою з рухомим початком**: **7 фолдів по 3 кроки**, навчальна частина розширюється.

У звіті наведіть:

1. таблицю MAE, RMSE і MAPE для всіх чотирьох базисів;
2. який базис виявився найкращим і **чому саме він** — поясніть через властивості вашого ряду;
3. для порівняння: ту саму оцінку, зроблену звичайним випадковим поділом 75/25, і **у скільки разів** вона відрізняється.

> 💡 **Підказка.** Кількість фолдів і кроків задана жорстко — у Частині 2 вона має збігтися з налаштуваннями Orange.

In [8]:
```
# Завдання 6. Базисні прогнози та ретроспективна перевірка
def calculate_metrics(y_true, y_pred):
    mae = np.mean(np.abs(y_true - y_pred))
    rmse = np.sqrt(np.mean((y_true - y_pred)**2))
    mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
    return mae, rmse, mape

num_folds = 7
h = 3
m = 12

total_test = num_folds * h
train_start = len(bread) - total_test

cv_results = {'Naive': [], 'SNaive': [], 'Mean': [], 'Drift': []}


for fold in range(num_folds):
    train_end = train_start + fold * h
    train = bread[:train_end]['ціна']
    test = bread[train_end : train_end + h]['ціна']

    #Naive
    f_naive = np.full(h,train.iloc[-1])

    #season naive
    f_season_naive = np.array([train.iloc[-m + (i % m)] for i in range(h)])

    #mean
    f_mean = np.full(h, np.mean(train))


    #Drift
    slope = (train.iloc[-1] - train.iloc[0]) / (len(train) - 1)
    f_drift = train.iloc[-1] + np.arange(1, h + 1) * slope
    cv_results['Naive'].append((test, f_naive))
    cv_results['SNaive'].append((test, f_season_naive))
    cv_results['Mean'].append((test, f_mean))
    cv_results['Drift'].append((test, f_drift))

summary = {}

for model_name, fold_list in cv_results.items():

    y_trues = np.concatenate([test_fold.values for test_fold, _ in fold_list])
    y_preds = np.concatenate([pred_fold for _, pred_fold in fold_list])


    mae, rmse, mape = calculate_metrics(y_trues, y_preds)
    summary[model_name] = {'MAE': round(mae, 3), 'RMSE': round(rmse, 3), 'MAPE (

df_metrics = pd.DataFrame(summary).T
print("=== ТАБЛИЦЯ МЕТРИК (Expanding CV 7x3) ===")
print(df_metrics)
```

```
train = bread[:len(bread)*75//100]['ціна']
test = bread[len(train):]['ціна']
h = len(test)
    #Naive
f_naive = np.full(h,train.iloc[-1])

    #season naive
f_season_naive = np.array([train.iloc[-m + (i % m)] for i in range(h)])

    #mean
f_mean = np.full(h, np.mean(train))


    #Drift
slope = (train.iloc[-1] - train.iloc[0]) / (len(train) - 1)
f_drift = train.iloc[-1] + np.arange(1, h + 1) * slope

split_results = {
    'Naive': (test.values, f_naive),
    'SNaive': (test.values, f_season_naive),
    'Mean': (test.values, f_mean),
    'Drift': (test.values, f_drift)
}

summary = {}

for model_name, (y_true_s, y_pred_s) in split_results.items():

    mae, rmse, mape = calculate_metrics(y_true_s, y_pred_s)
    summary[model_name] = {'MAE': round(mae, 3), 'RMSE': round(rmse, 3), 'MAPE (

df_metrics = pd.DataFrame(summary).T
print("=== ТАБЛИЦЯ МЕТРИК (Expanding CV 7x3) ===")
print(df_metrics)
```

```
=== ТАБЛИЦЯ МЕТРИК (Expanding CV 7x3) ===
          MAE   RMSE  MAPE (%)
Naive   0.586  0.797      6.28
SNaive  1.029  1.274     10.19
Mean    2.137  2.261     21.69
Drift   0.616  0.873      6.61
=== ТАБЛИЦЯ МЕТРИК (Expanding CV 7x3) ===
          MAE   RMSE  MAPE (%)
Naive   0.572  0.691      5.46
SNaive  1.254  1.520     12.01
Mean    2.617  2.681     25.60
Drift   1.049  1.256     10.76
```

Висновок до завдання 6:

сезонний наївний показав гірші результати бо за рік ціна збільшилась.Найкращим є наївний прогноз. для розподілу 75 25 оцінки збільшились у 0.3 рази

Завдання 7. Модель ARIMA

Побудуйте модель ARIMA і оцініть її **на тих самих 7 фолдах по 3 кроки**.

Обов'язково використайте порядок **(1, 1, 1)** — саме його ви відтворюватимете в Orange. Додатково визначте, який порядок (p, d, q) ви обрали б **за виглядом ACF і PACF** із завдання 5.

У звіті наведіть:

1. MAE, RMSE і MAPE моделі ARIMA(1,1,1) на рухомому початку;
2. обґрунтування порядку, який ви обрали б самостійно, з посиланням на конкретні лаги;
3. **порівняння з найкращим базисом із завдання 6** — і прямий висновок, чи має ця модель право на існування;
4. графік залишків моделі та висновок, чи лишилася в них структура.

> 💡 **Підказка.** Якщо ARIMA програла базису — це законний результат, а не помилка. Напишіть про це прямо.

In [9]:
```
# Завдання 7. Модель ARIMA
from statsmodels.tsa.arima.model import ARIMA

num_folds = 7
h = 3
m = 12

total_test = num_folds * h
train_start = len(bread) - total_test



test_r = []
pred = []
for fold in range(num_folds):
    train_end = train_start + fold * h
    train = bread[:train_end]['ціна']
    test = bread[train_end : train_end + h]['ціна']

    model = ARIMA(train,order = (1,1,1))

    model = model.fit()

    res=model.forecast(steps=3)
    test_r.extend(test.values)
    pred.extend(res.values)
    print(res)


test_r = np.array(test_r)
pred = np.array(pred)

mae_a, rmse_a, mape_a = calculate_metrics(test_r, pred)
print("mae,rmse,mape" \
"",mae_a,rmse_a,mape_a)
```

```
25    8.470116
26    8.470197
27    8.470253
Name: predicted_mean, dtype: float64
28    9.433279
29    9.511609
30    9.564240
Name: predicted_mean, dtype: float64
31    9.837391
32    9.899665
33    9.944040
Name: predicted_mean, dtype: float64
34    9.854304
35    9.795505
36    9.828651
Name: predicted_mean, dtype: float64
37    10.708671
38    10.721408
39    10.717482
Name: predicted_mean, dtype: float64
40    10.757794
41    10.755403
42    10.756137
Name: predicted_mean, dtype: float64
43    9.390714
44    9.409405
45    9.404363
Name: predicted_mean, dtype: float64
mae,rmse,mape 0.5887336964788226 0.8070431015109606 6.33002432662005
c:\Users\Acer\project_5_term\.venv\Lib\site-packages\statsmodels\tsa\statespace\m
lemodel.py:679: EstimationWarning: Non-stationary starting autoregressive paramet
ers found. Using zeros as starting parameters.
  start_params = self.start_params
c:\Users\Acer\project_5_term\.venv\Lib\site-packages\statsmodels\tsa\statespace\m
lemodel.py:679: EstimationWarning: Non-invertible starting MA parameters found. U
sing zeros as starting parameters.
  start_params = self.start_params
```

Висновок до завдання 7:

я би обрав 1 1 4 бо після 4 падає в зону ARIMA трошки програє наївному прогнозу

Завдання 8. Точка зміни режиму

Знайдіть у своєму ряді точку, після якої змінюється **рівень** ряду. Скористайтеся скануванням: для кожної кандидатної дати порівняйте середні до і після та оберіть дату з найбільшою t-статистикою.

У звіті наведіть:

1. графік ряду з позначеною знайденою датою;
2. дату, значення t-статистики, середні рівні до і після;
3. **чи є ця точка значущою** — і якщо ні, так і напишіть;
4. чим знайдена точка відрізняється від звичайного сезонного сплеску у вашому ряді.

> 💡 **Підказка.** Не плутайте зсув рівня з сезонним піком. Якщо «аномалії» повторюються щороку в ті самі місяці — це сезон.

In [10]:
```
# Завдання 8. Точка зміни режиму
import math

plt.plot(bread.index,bread['ціна'])
start = 6
end = 40
t = [-math.inf]*len(bread)
for k in range(start,end):
    before = bread['ціна'][:k]
    after =  bread['ціна'][k:]
    t_k = (before.mean() - after.mean())/math.sqrt(np.var(before)/len(before) +
    t[k] = math.fabs(t_k)
    print(k,before.mean(),after.mean(),t_k)
res_t = np.argmax(t)
print("k  data  t_k  before  after")
print(np.argmax(t),bread.loc[np.argmax(t)]['ts'],t[np.argmax(t)],bread['ціна'][:
plt.scatter(12,bread.loc[12]['ціна'],colorizer="red")
plt.axvline(x = 12,color="red",linestyle="--")
plt.axvline(x = 24,color="red",linestyle="--")
plt.axvline(x = 36,color="red",linestyle="--")




real_max_t = t[res_t]


n_permutations = 1000
perm_max_t = []

prices = bread['ціна'].values

for _ in range(n_permutations):
    shuffled_prices = np.random.permutation(prices)
    current_perm_t = []
    for k in range(start, end):
        before = shuffled_prices[:k]
        after = shuffled_prices[k:]

        se = math.sqrt(before.var() / len(before) + after.var() / len(after))
        if se > 0:
            t_k = abs((before.mean() - after.mean()) / se)
        else:
            t_k = 0
        current_perm_t.append(t_k)

    perm_max_t.append(max(current_perm_t))

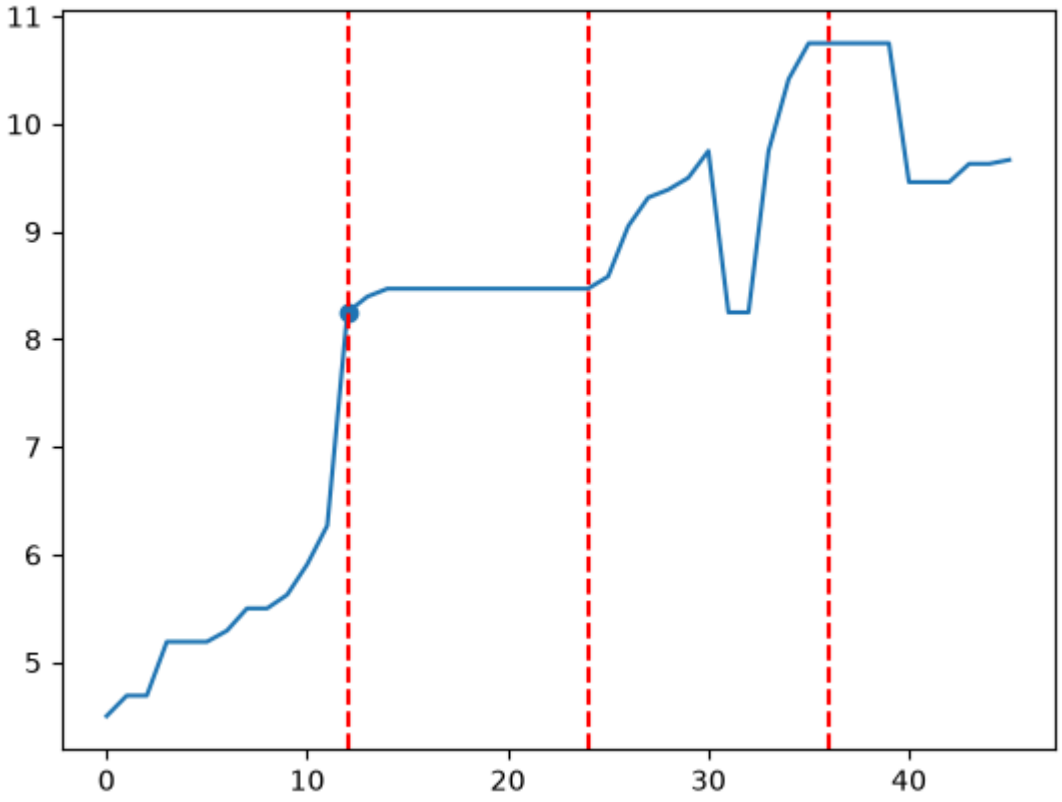
p_value = np.mean(np.array(perm_max_t) >= real_max_t)

print(f"Емпіричне p-value: {p_value:.4f}")

if p_value < 0.05:
    print("Результат: Точка зміни є СТАТИСТИЧНО ЗНАЧУЩОЮ (p < 0.05)")
```

```
else:
    print("Результат: Точка зміни НЕ Є значущою (це випадковий шум)")
```

```
6 4.908333333333334 8.6807525 -14.328524488476317
7 4.963328571428572 8.767610256410254 -15.112953625247071
8 5.030412500000001 8.853599999999997 -15.68536540089621
9 5.082588888888889 8.944237837837836 -16.771992299982994
10 5.1370000000000005 9.036391666666667 -18.050998646096975
11 5.207272727272727 9.125717142857141 -18.996963826369942
12 5.295833333333333 9.209708823529411 -19.190478700628127
13 5.523076923076923 9.238790909090909 -12.619018475023374
14 5.728335714285714 9.265106249999999 -10.315847068326848
15 5.911113333333333 9.290754838709677 -9.122301330610211
16 6.07104375 9.318113333333335 -8.433924755006315
17 6.212158823529412 9.347358620689654 -8.000026169195284
18 6.337594444444445 9.378692857142857 -7.716399364153923
19 6.4498263157894735 9.412348148148148 -7.531712804887762
20 6.550835000000001 9.448592307692307 -7.418052299764917
21 6.64222380952381 9.487736 -7.359498528356536
22 6.725304545454546 9.530141666666667 -7.347102563865378
23 6.8011608695652175 9.576234782608697 -7.376557206613023
24 6.8706958333333334 9.626518181818183 -7.447216142328388
25 6.934668 9.681590476190475 -7.561973301312647
26 6.9980769230769235 9.736505 -7.663486311134219
27 7.074074074074075 9.772636842105262 -7.554317851025547
28 7.154167857142858 9.797966666666667 -7.344507154611841
29 7.23126551724138 9.821964705882353 -7.1312545379956855
30 7.30689 9.842087500000002 -6.892470359638183
31 7.385700000000001 9.848226666666667 -6.56702255618929
32 7.412709375 9.962385714285714 -7.123850789010178
33 7.438081818181819 10.094107692307691 -7.9857074289931145
34 7.5060794117647065 10.122783333333334 -7.765275866972363
35 7.58924 10.096063636363636 -7.235033432251333
36 7.677038888888889 10.03067 -6.658302161279354
37 7.760091891891892 9.950744444444446 -6.12350472947615
38 7.838773684210526 9.8508375 -5.632653392006915
39 7.913420512820513 9.722385714285716 -5.221707045950442
k  data  t_k  before  after
12 2015-03-01 00:00:00 19.190478700628127 5.295833333333333 9.209708823529411
Емпіричне p-value: 0.0000
Результат: Точка зміни є СТАТИСТИЧНО ЗНАЧУЩОЮ (p < 0.05)
```



Висновок до завдання 8:

знайдено значущу точку методом сканування і перевірено значущість завдяки p-val < 0.05. це не сезонність бо нема чіткої періодичності в піднятті і нема повернення назад ціни

Завдання 9. Рецензія на чужий аналіз

Відкрийте `ЛР4_помилковий_аналіз.ipynb`. Це звіт, зроблений на іншому ряді з того самого набору. Він виконується без помилок і виглядає переконливо, але містить **методологічні помилки**.

У звіті наведіть:

1. перелік знайдених помилок — для кожної вкажіть номер клітинки, суть помилки й **чим вона небезпечна**;
2. для трьох найсерйозніших — як саме їх виправити.

> 💡 **Підказка.** Помилок щонайменше вісім. Шукайте не синтаксис, а порушення принципів роботи з часовими рядами.

In [11]:
```
# Завдання 9. Рецензія на чужий аналіз
"""
2 розділ порівняли ціну 14 і 17 року і сказали про купівельну спровожність але н
3 розідл порівгяли тільки MAE на цінах різного маштабу тре було брати MNSE
4 розділ вислока кореляція не означає шо одне впливає на інше, може інший чинник
5 розділ STL завжди визначає сезонну складову але вона може складатись з шуму пр
6 розділ ряд є нестаціонарним
7 розділ не враховано сезонність
```

```
8 розділ розділено рандомним 75/25 задача стала інтерполяцією
10 розділ обрізано часовий ряд, півріччя могло припасти на сезонність тому більщ
"""
```

Out[11]:
```
'\n2 розділ порівняли ціну 14 і 17 року і сказали про купівельну спровожність а
ле не врахували рівень цін і зарплати\n3 розідл порівгяли тільки MAE на цінах р
ізного маштабу тре було брати MNSE\n4 розділ вислока кореляція не означає шо од
не впливає на інше, може інший чинник вплинув на обидва стани\n5 розділ STL зав
жди визначає сезонну складову але вона може складатись з шуму при силі сезоннос
ті < 0.3\n6 розділ ряд є нестаціонарним\n7 розділ не враховано сезонність\n8 ро
зділ розділено рандомним 75/25 задача стала інтерполяцією\n10 розділ обрізано ч
асовий ряд, півріччя могло припасти на сезонність тому більщилось\n'
```

Висновок до завдання 9:

(напишіть тут)

Завдання 10. Експорт ряду для Orange

Збережіть підготовлений ряд у файл `ряд_для_orange.csv` **рівно у такому вигляді**:

Стовпець	Тип	Зміст
`date`	текст у форматі `РРРР-ММ-ДД`	перше число місяця
`price`	число	ціна, пропуск уже заповнений

Один рядок на місяць, 46 рядків плюс заголовок, роздільник — кома, кодування UTF-8.

У звіті наведіть:

1. підтвердження, що файл має 46 рядків даних і не містить порожніх значень.

> 💡 **Підказка.** Формат заданий жорстко: Orange має отримати **точно ті самі** дані, що й Python, інакше порівняння в Частині 2 втрачає сенс.

In [14]:
```
# Завдання 10. Експорт ряду для Orange
bread[['ts','ціна']].to_csv("ряд_для_orange.csv")
export = bread[['ts','ціна']]
print(len(export),export.notna().sum())
```

```
46 ts      46
ціна    46
dtype: int64
```

Висновок до завдання 10:

(напишіть тут)

ЧАСТИНА 2. Той самий аналіз засобами Orange

Виконується самостійно.

Потрібен додаток **Timeseries**: у Orange відкрийте *Options → Add-ons…* і встановіть `Orange3-Timeseries`.

Головна умова цієї частини: Orange має отримати **ті самі дані** й **ті самі налаштування перевірки**, що й Python у Частині 1. Тоді будь-яка розбіжність у числах пояснюється **лише означеннями метрик**, а не різними даними чи різними моделями. Саме це вам і треба довести.

Завдання 11. Потік опрацювання даних

Побудуйте в Orange потік:

`File` → `As Timeseries` → `Interpolate` → `Line Chart`

У віджеті **File** завантажте свій `ряд_для_orange.csv`; переконайтеся, що `date` розпізнано як **time**, а `price` — як **numeric**. У віджеті **As Timeseries** оберіть `date` як часову змінну.

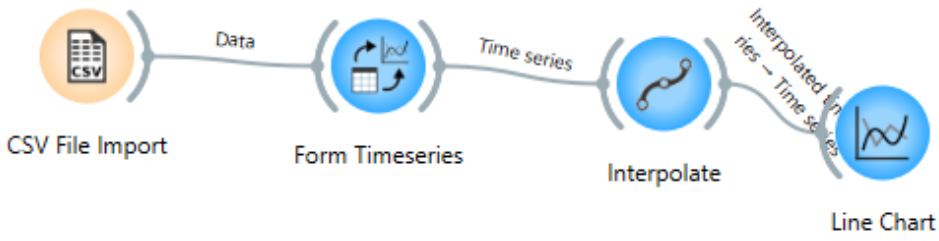
У звіті наведіть:

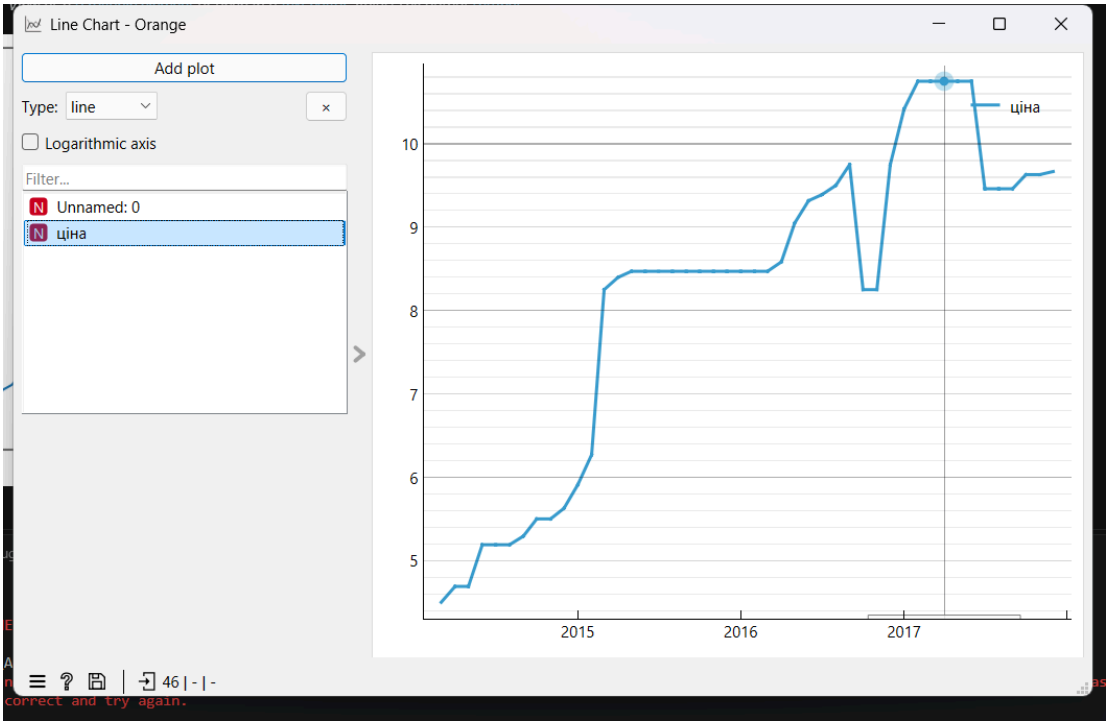
1. знімок екрана з побудованим потоком;
2. знімок `Line Chart` із вашим рядом;
3. підтвердження, що графік у Orange збігається з графіком із завдання 3.

> 💡 **Підказка.** Якщо `date` розпізнано як текст, поверніться до завдання 10 і перевірте формат дати.

In [ ]:
```
# Завдання 11. Потік опрацювання даних
```

Висновок до завдання 11:





Завдання 12. Автокореляція та диференціювання

Додайте до потоку віджети `Correlogram`, `Difference` та `Seasonal Adjustment`.

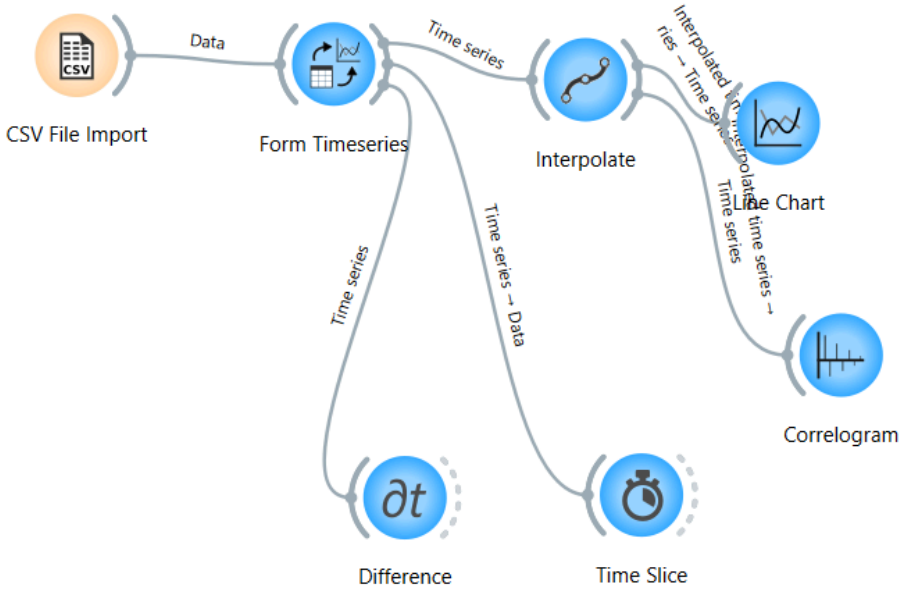
У звіті наведіть:

1. знімок корелограми;
2. зіставлення з ACF із завдання 5: **чи збігаються висновки** про значущі лаги;
3. знімок сезонно скоригованого ряду та порівняння з сезонною складовою STL із завдання 4.

> 💡 **Підказка.** Orange і statsmodels рахують автокореляцію однаково — якщо картина різна, перевірте, чи однакові дані на вході.

In [ ]:
```
# Завдання 12. Автокореляція та диференціювання
```

Висновок до завдання 12:



Завдання 13. Модель і оцінювання

Додайте `ARIMA Model` з порядком **(1, 1, 1)** та `Model Evaluation`.

У віджеті **Model Evaluation** встановіть **Number of folds = 7** та **Forecast steps = 3**.

У звіті наведіть:

1. знімок таблиці Model Evaluation з усіма метриками;
2. **що станеться, якщо лишити типові налаштування** (20 фолдів × 3 кроки) — наведіть повідомлення, яке видасть Orange, і поясніть, звідки воно береться;
3. які саме 7 фолдів перевіряє Orange — тобто на яких відрізках ряду навчається модель.

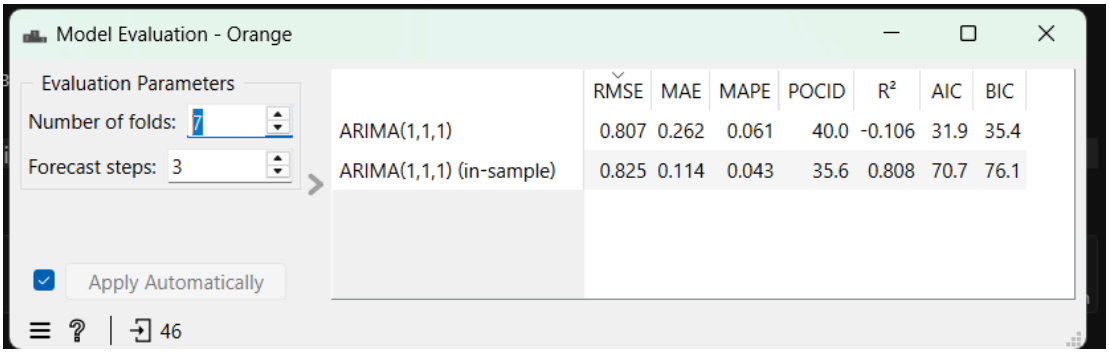
> 💡 **Підказка.** Orange відмовиться працювати при типових налаштуваннях. Це не збій, а наслідок довжини вашого ряду — порахуйте, якої саме.

In [ ]:
```
# Завдання 13. Модель і оцінювання
```

Висновок до завдання 13:

Unexpected error: Supplied time series is too short for this many folds / step size. Retry with fewer iterations. даних 45 16 фолдів вже не рахує тому шо даних потрібно шоб

16 раз порахувати 48 а їх 45



Завдання 14. Чому числа не збігаються

Порівняйте метрики Orange з метриками, які ви отримали в Частині 1 для **тієї самої** моделі ARIMA(1,1,1) на **тих самих** 7 фолдах.

Для кожної з чотирьох метрик Orange — `RMSE`, `MAE`, `MAPE`, `POCID` — **відтворіть її значення в Python**, тобто напишіть формулу, за якою Orange отримав саме це число, і переконайтеся, що ваш розрахунок збігається з таблицею Orange до третього знака.

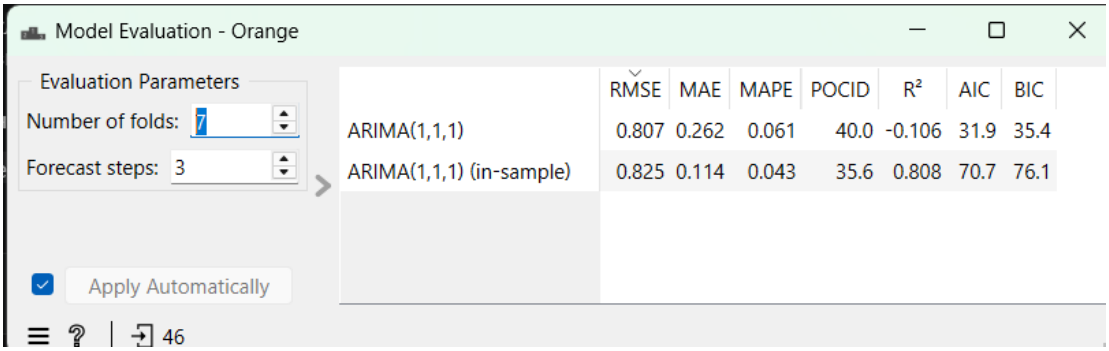
У звіті наведіть:

1. таблицю на чотири рядки: метрика · значення Orange · ваше відтворення в Python · формула;
2. **яка з метрик не відповідає своїй назві** — і чим саме вона відрізняється від того, що зазвичай позначають цією абревіатурою;
3. яка метрика збіглася між середовищами одразу і чому;
4. висновок: що саме треба перевіряти, перш ніж порівнювати числа з двох різних інструментів.

> 💡 **Підказка.** Жодна з розбіжностей не є помилкою Orange. Усі формули відкриті — шукайте їх у вихідному коді додатка Timeseries, файл `functions.py`.

In [ ]:
```
# Завдання 14. Чому числа не збігаються
```

Висновок до завдання 14:



python mae,rmse,mape 0.5887336964788226 0.8070431015109606 6.33002432662005

метрика	Orange	Python	формула
MAE	0.262	0.588	середнє це сума всіх похибок/n а медіанне це медіана похибок
RMSE	0.807	0.807	корінь суми квадратів похибок/n
MAPE	6.110	6.330	середнє суми модулів похибки поділена на реальне значення, в пайтоні ше помножене на 100 для відсотків

Завдання 15. Матриця напрямку зміни

Метрика `POCID` у Orange вимірює, чи правильно модель передбачає **напрямок** зміни ціни. Розгорніть її в таблицю 2×2: рядки — що сталося насправді (зросла / впала), стовпці — що передбачила модель.

У звіті наведіть:

1. заповнену матрицю 2×2 для вашої моделі;
2. перевірку, що частка правильних передбачень збігається зі значенням POCID із Orange;
3. висновок: чи корисна ваша модель для рішення «купувати зараз чи чекати», і чому це **інше питання**, ніж те, на яке відповідають MAE та RMSE.

> 💡 **Підказка.** Ця матриця — часоворядовий відповідник матриці похибок із класифікації.

In [ ]:
```
from statsmodels.tsa.arima.model import ARIMA
import pandas as pd
from sklearn.metrics import confusion_matrix

num_folds = 7
h = 3
m = 12

total_test = num_folds * h
train_start = len(bread) - total_test

test_r = []
pred = []


all_actual_classes = []
all_predicted_classes = []

for fold in range(num_folds):
    train_end = train_start + fold * h
    train = bread[:train_end]['ціна']
    test = bread[train_end : train_end + h]['ціна']

    model = ARIMA(train, order=(1,1,1))
    model = model.fit()

    res = model.forecast(steps=3)
    test_r.extend(test.values)
```

```
    pred.extend(res.values)
    print(res)


    prev_val = train.values[-1]

    fold_test_with_prev = np.insert(test.values, 0, prev_val)
    fold_pred_with_prev = np.insert(res.values, 0, prev_val)

    actual_change = fold_test_with_prev[1:] - fold_test_with_prev[:-1]
    predicted_change = fold_pred_with_prev[1:] - fold_test_with_prev[:-1]

    actual_class = (actual_change > 0).astype(int)
    predicted_class = (predicted_change > 0).astype(int)

    all_actual_classes.extend(actual_class)
    all_predicted_classes.extend(predicted_class)


test_r = np.array(test_r)
pred = np.array(pred)

mae_a, rmse_a, mape_a = calculate_metrics(test_r, pred)
print("mae,rmse,mape:", mae_a, rmse_a, mape_a)



cm = confusion_matrix(all_actual_classes, all_predicted_classes, labels=[1, 0])

matrix_df = pd.DataFrame(
    cm,
    index=['Фактично Зросло', 'Фактично Впало'],
    columns=['Передбачено Зросте', 'Передбачено Впаде']
)

total_hits = np.sum(np.array(all_actual_classes) == np.array(all_predicted_class
pocid_calc = (total_hits / len(all_actual_classes)) * 100

print("\n=== Матриця напрямку зміни (Завдання 15) ===")
print(matrix_df)
print(f"Розрахований POCID: {pocid_calc:.1f}%")
```

```
25    8.470116
26    8.470197
27    8.470253
Name: predicted_mean, dtype: float64
28    9.433279
29    9.511609
30    9.564240
Name: predicted_mean, dtype: float64
31    9.837391
32    9.899665
33    9.944040
Name: predicted_mean, dtype: float64
34    9.854304
35    9.795505
36    9.828651
Name: predicted_mean, dtype: float64
37    10.708671
38    10.721408
39    10.717482
Name: predicted_mean, dtype: float64
40    10.757794
41    10.755403
42    10.756137
Name: predicted_mean, dtype: float64
43    9.390714
44    9.409405
45    9.404363
Name: predicted_mean, dtype: float64
mae,rmse,mape: 0.5887336964788226 0.8070431015109606 6.33002432662005

=== Матриця напрямку зміни (Завдання 15) ===
                 Передбачено Зросте  Передбачено Впаде
Фактично Зросло                   6                  5
Фактично Впало                    5                  5
Розрахований POCID: 52.4%
c:\Users\Acer\project_5_term\.venv\Lib\site-packages\statsmodels\tsa\statespace\m
lemodel.py:679: EstimationWarning: Non-stationary starting autoregressive paramet
ers found. Using zeros as starting parameters.
  start_params = self.start_params
c:\Users\Acer\project_5_term\.venv\Lib\site-packages\statsmodels\tsa\statespace\m
lemodel.py:679: EstimationWarning: Non-invertible starting MA parameters found. U
sing zeros as starting parameters.
  start_params = self.start_params
```

Висновок до завдання 15:

mse rmse відповідають на питання на скільки одиниць є похибка, а посід на те чи визначено рух графіка

Загальний висновок

Напишіть 8–12 речень, які відповідають на такі питання:

1. Чи є у вашому ряді сезонність і чим ви це довели?

2. Який метод прогнозування виявився найкращим і чи перевершив він найпростіший базис?
3. Чи знайшли ви точку зміни режиму; якщо так — чи є для неї зовнішнє пояснення?
4. Чому однаково названі метрики у двох середовищах дали різні числа?
5. Чого ви не змогли встановити через те, що ряд має лише 46 спостережень?

1.сезонність є довів силою сезонності 2.Аріма 3.знайшов, це точка не є сезонністю, можливо шось інше з ціною сталось 4.бо одна рахує середнє друга медіану 5.репрезентаривну оцінку бо фолдів мало

Критерії оцінювання

Бали	Умова
0	Ряд не розкладено або прогноз не побудовано. Робота не зарахована.
2	Виконано лише Частину 1: ряд розкладено й прогноз побудовано, але в Orange аналіз не відтворено.
3	Виконано обидві частини, проте прогноз не зіставлено з наївним базисом, а розбіжність результатів двох середовищ не пояснено.
4	Ряд розкладено на складові; прогноз зіставлено з наївним базисом за ретроспективною перевіркою з рухомим початком; той самий аналіз відтворено в Orange; зіставлено результати двох середовищ і пояснено розбіжність метрик.

Знімаються бали за:

- графіки без пояснення вибору геометрії та шкали;
- числа без письмового висновку;
- прогноз без порівняння з базисом;
- висновок «метрики різні, бо середовища різні» без відтворення формул;
- зошит, який не виконується від початку до кінця.

Декларація про використання штучного інтелекту

Використання інструментів штучного інтелекту під час виконання цієї роботи **дозволено**. Умова одна: ви маєте розуміти кожен рядок зданого коду й кожне написане твердження і бути готовим пояснити їх на захисті.

Заповніть таблицю. Якщо ШІ не використовувався, напишіть «не використовувався» в першому рядку й залиште решту порожньою.

Питання	Ваша відповідь
Які інструменти використано (назва, версія)	gemini
Для яких саме завдань	генерація коду для синтаксису операцій з бібліотеками
Що саме було згенеровано (код / текст / пояснення / виправлення помилок)	код, пояснення
Що ви змінили у згенерованому результаті й чому	нічого
Що з отриманого ви **перевірили самостійно** і яким способом	нічого
Чи траплялися у відповідях ШІ помилки; якщо так — які	сказав шо в оранджі має це середнє

Підтверджую, що розумію весь зданий код і всі зроблені висновки та можу пояснити будь-яку їх частину на захисті роботи.

Підпис: + Дата: 02.11.26